

UNO - Progetto MDP 2025/26

Osema Fadhel (Matricola 2225298)

0. Introduzione.....	2
1. Analisi del Progetto e Scelte Interpretative.....	2
1.1 Modalità di partita implementate.....	2
1.2 Gestione del punteggio e delle penalità.....	2
1.3 Semplificazioni e scelte interpretative adottate.....	3
2. Architettura del Software.....	3
2.1 Architettura complessiva.....	3
2.2 Design Pattern utilizzati.....	4
2.3 Diagramma UML delle classi principali.....	4
2.4 Descrizione delle responsabilità delle classi.....	6
3. Dettagli Implementativi.....	7
3.1 Classi astratte.....	7
3.2 Interfacce.....	8
3.3 Enumerazioni.....	9
3.4 Lambda, Comparatori.....	10
3.5 Gestione errori.....	11
4. Interfaccia Grafica (GUI).....	12
4.1 Menu principale.....	12
4.2 Schermata di gioco.....	15
4.3 Fasi speciali.....	15
4.4 Fine round e fine partita.....	16
5. Strategie dei Bot.....	18
5.1 Bot Casuale (RandomBotStrategy).....	18
5.2 Bot Greedy (GreedyBotStrategy).....	18
6. Istruzioni per l'uso.....	19
Prerequisiti.....	19
Da terminale.....	19
Da Eclipse.....	19
7. Limiti Noti e Conclusioni.....	20

0. Introduzione

Il progetto consiste nella progettazione e nello sviluppo in Java del gioco di carte UNO. L'obiettivo è implementare una versione funzionante del gioco con interfaccia grafica, supportando partite tra giocatori umani e bot.

UNO è un gioco di carte in cui l'obiettivo è liberarsi di tutte le carte in mano prima degli avversari. Ciascun giocatore, nel proprio turno, può giocare una carta compatibile con quella in cima alla pila degli scarti oppure pescarne una dal mazzo. Il mazzo è composto da 108 carte suddivise in carte numeriche, carte azione e carte jolly, ciascuna con regole di gioco e punteggi specifici.

Quando un giocatore rimane con una sola carta in mano deve dichiarare UNO; in caso contrario, può essere contestato dagli altri e costretto a pescare come penalità.

Il progetto implementa la versione base del gioco, con supporto a partite tra due e sei giocatori umani o bot, due modalità di partita e un'interfaccia grafica interattiva.

1. Analisi del Progetto e Scelte Interpretative

1.1 Modalità di partita implementate.

Il progetto supporta due modalità di partita selezionabili nella configurazione iniziale.

Nella partita singola, il gioco si svolge in un unico round. Vince il primo giocatore che esaurisce tutte le carte in mano. Al termine della partita viene calcolato e mostrato il punteggio ottenuto dal vincitore, pari alla somma del valore delle carte rimaste in mano agli avversari, secondo le regole di conteggio classiche.

Nella partita a punti, il gioco è composto in più round consecutivi. Al termine di ogni round il vincitore accumula punti pari al valore delle carte degli altri giocatori. I punti si sommano tra un round e l'altro fino a quando un giocatore raggiunge o supera la soglia di vittoria configurata nel menu principale, il cui valore predefinito è 500.

1.2 Gestione del punteggio e delle penalità.

Il punteggio di fine round segue le regole classiche: le carte numeriche valgono il proprio valore, le carte azione (Skip, Reverse, Draw Two) valgono 20 punti ciascuna, le carte jolly (Wild e Wild Draw Four) valgono 50 punti ciascuna.

Durante la partita sono gestite le seguenti penalità:

- Il giocatore colpito da un Draw Two pesca due carte e perde il turno.
- Il giocatore colpito da un Wild Draw Four pesca quattro carte e perde il turno, salvo che decida di effettuare un challenge. Se il challenge ha successo, il giocatore che ha giocato il Wild Draw Four illegalmente pesca quattro carte; se fallisce, il challenger pesca sei carte.
- Se un giocatore non dichiara UNO quando rimane con una sola carta in mano e viene contestato con successo, pesca due carte come penalità.

1.3 Semplificazioni e scelte interpretative adottate.

Il progetto include alcune semplificazioni e scelte interpretative rispetto alle regole originali del gioco, descritte di seguito.

All'avvio di ogni round, la prima carta visibile sul mazzo degli scarti viene estratta fino a ottenerne una non jolly, in modo da garantire un colore di partenza definito. L'effetto eventuale di questa carta (Skip, Reverse e Draw Two) non viene applicato: il turno parte sempre dal primo giocatore senza modifiche.

Un giocatore umano può dichiarare UNO premendo il relativo pulsante quando ha due carte in mano. Nei bot, la dichiarazione avviene automaticamente al momento del gioco della penultima carta. La finestra di contestazione è il turno del giocatore successivo: se un bot rileva che il giocatore precedente ha una sola carta senza aver dichiarato UNO, può contestarlo prima di eseguire la propria mossa, in base alla propria strategia.

Con due giocatori, quando viene giocata una carta Reverse, l'effetto produce lo stesso ordine di turni di quello originale. Si è scelto di implementare la Reverse in modo uniforme per qualsiasi numero di giocatori, con l'effetto che, con due giocatori, la carta non produce alcun cambiamento percepibile sull'ordine di turno.

La specifica prevede la modalità privacy solo per i gruppi da tre studenti; per il progetto individuale non è richiesta. Tuttavia, si è deciso di nascondere la mano del giocatore corrente per i primi tre secondi di ogni turno, al fine di concedere al giocatore precedente il tempo di allontanarsi prima che le proprie carte vengano mostrate.

2. Architettura del Software

2.1 Architettura complessiva.

Il progetto è strutturato secondo il pattern architetturale Model-View-Controller (MVC), che separa la logica di gioco dalla rappresentazione grafica e dalla gestione degli eventi utente. Essendo pensata come applicazione interattiva, con un menù principale e una serie di bottoni nelle varie view, risulta efficace avere un controller che intercetta gli eventi della view, apportando, se necessario, modifiche al Model e aggiornando di conseguenza la view.

Il Model comprende tutte le classi del package *it.uniroma1.mdp.uno.model* e dei suoi sottopackages. Gestisce lo stato della partita, modella le carte e le loro regole, i giocatori, e le strategie dei bot.

La View è composta da due classi del package *it.uniroma1.mdp.uno.view*:

- La classe Menu gestisce la schermata di configurazione iniziale.
- La classe Game gestisce il tavolo di gioco con tutti i suoi overlay.

Le classi della view costruiscono l'interfaccia grafica e espongono i propri componenti interattivi tramite metodi getter.

Il Controller, composto dalla classe GameController nel package *it.uniroma1.mdp.uno.controller*, si occupa di invocare i metodi del model in risposta alle interazioni dell'utente con la GUI, e di aggiornare la view in base al nuovo stato. Gestisce inoltre il turno di gioco dei bot.

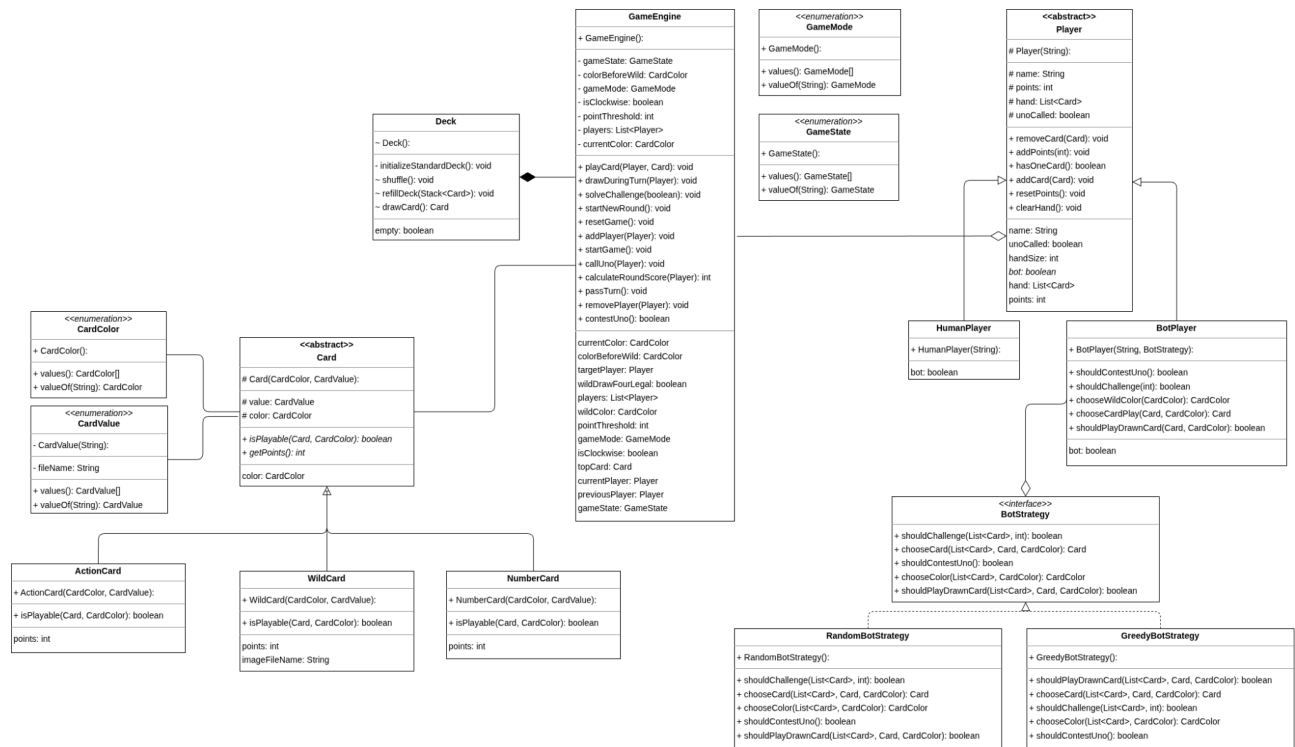
2.2 Design Pattern utilizzati

Il progetto fa uso di tre design pattern:

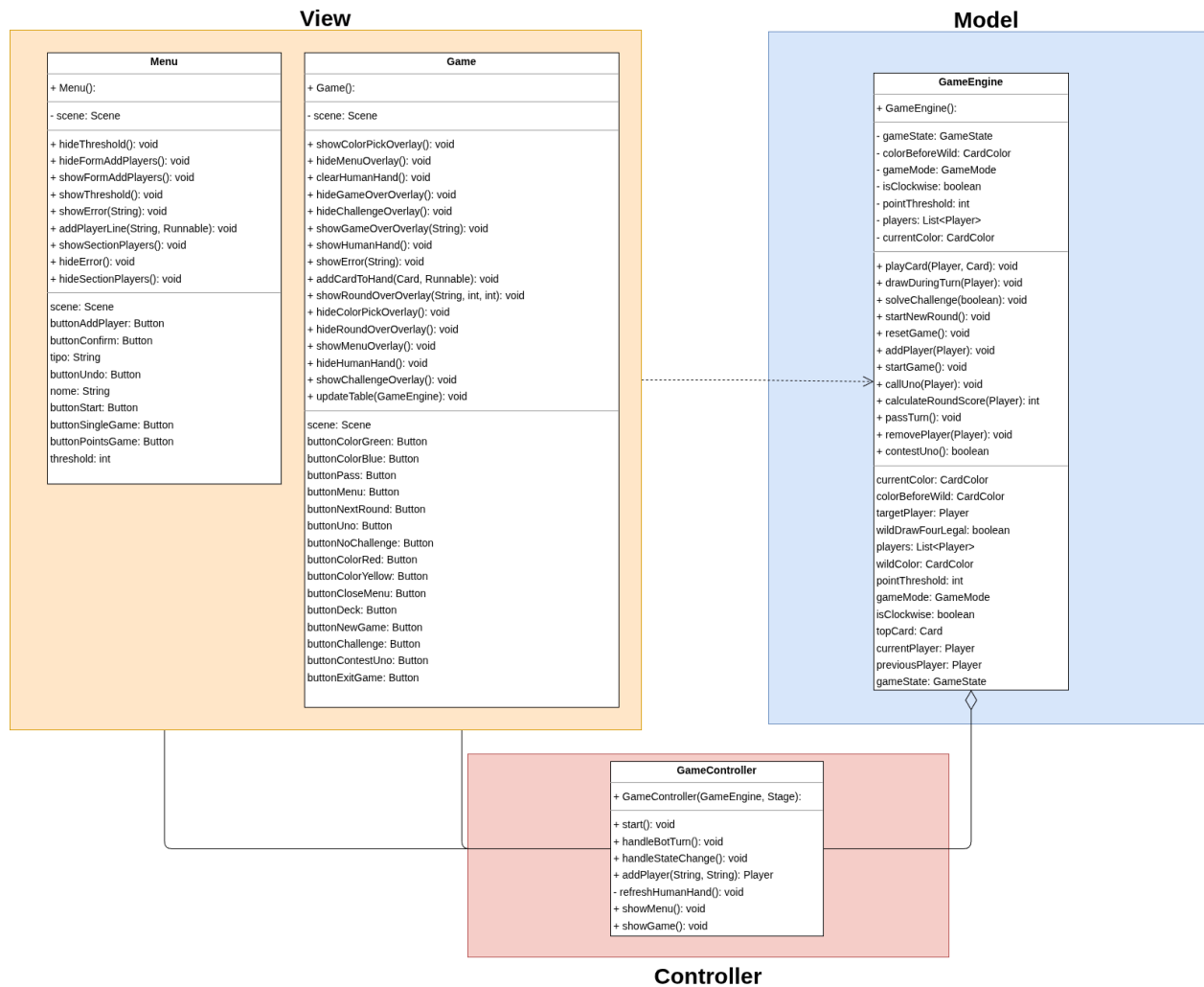
- State: L'enum GameState definisce sei stati distinti (es. WAITING_FOR_PLAYER_ACTION, ROUND_OVER...). Ogni metodo pubblico in GameEngine, verifica lo stato corrente della partita prima di agire, garantendo così che le regole del gioco siano rispettate.
- Template Method: Le classi astratte Card e Player definiscono la struttura comune alle rispettive gerarchie e dichiarano astratti i metodi il cui comportamento varia tra le sottoclassi: getPoints() e isPlayable() in Card, isBot() in Player. Le sottoclassi concrete (NumberCard, ActionCard, WildCard, HumanPlayer, BotPlayer) forniscono l'implementazione specifica, rimuovendo duplicazione di codice.
- Strategy: Il comportamento dei bot è separato dalla loro struttura tramite l'interfaccia BotStrategy. La classe BotPlayer delega tutte le decisioni all'istanza di strategia iniettata alla creazione, rendendo i profili comportamentali intercambiabili senza modificare la classe BotPlayer.

2.3 Diagramma UML delle classi principali.

Il seguente diagramma UML delle classi illustra la struttura del livello Model:



Il secondo diagramma mostra l'architettura complessiva dell'applicazione secondo il pattern MVC, evidenziando le dipendenze tra i tre layer:



2.4 Descrizione delle responsabilità delle classi.

Il motore centrale dell'applicazione è GameEngine: gestisce lo stato dell'intera partita, applica le regole a ogni azione dei giocatori, calcola i punteggi e coordina l'avanzamento dei turni. Tutta la logica di gioco risiede in questa classe.

La classe Deck si occupa della creazione del mazzo di 108 carte standard, del mescolamento e del rifornimento del mazzo dalla pila degli scarti quando le carte si esauriscono.

Le carte sono modellate attraverso una gerarchia con radice nella classe astratta Card, che definisce i campi comuni (colore e valore) e i metodi astratti getPoints() e isPlayable(). Le tre sottoclassi, NumberCard, ActionCard e WildCard, forniscono l'implementazione specifica per ciascun tipo di carta.

I giocatori sono rappresentati da una gerarchia con radice nella classe astratta Player, che gestisce la mano di carte, i punti e la dichiarazione UNO. La sottoclasse HumanPlayer non aggiunge logica propria,

mentre la sottoclasse BotPlayer ha un riferimento a una BotStrategy, che delega a quest'ultima tutte le decisioni di gioco. BotStrategy è l'interfaccia che definisce il comportamento dei bot, che ha due implementazioni: RandomBotStrategy e GreedyBotStrategy.

La View è composta da due classi. Menu gestisce la schermata di configurazione iniziale, permettendo di scegliere la modalità di partita, aggiungere giocatori e impostare la soglia di punti per la modalità a punti. La classe Game gestisce invece la schermata di gioco: mostra il tavolo di gioco, la mano del giocatore corrente e i bottoni di azione, e gestisce la visibilità degli overlay per le fasi speciali (scelta colore, challenge, fine round, fine partita). Entrambe le classi di view non contengono alcuna logica di gioco ed espongono i propri componenti tramite getter.

Il GameController è l'unica classe che conosce sia il Model sia la View. Il controller associa le funzioni lambda come gestori degli eventi sui bottoni esposti dai getter. A ogni interazione dell'utente, invoca il corrispondente metodo del motore e aggiorna la view tramite handleStateChange().

3. Dettagli Implementativi

3.1 Classi astratte

Il progetto fa uso di due classi astratte: Card e Player. Entrambe seguono lo stesso principio, quello di definire nella classe base tutto ciò che è comune alle sottoclassi, e dichiarare astratto solo ciò che varia.

La classe Card è la superclasse da cui derivano tutte le altre carte. I metodi getPoints() e isPlayable() sono dichiarati astratti perché ogni tipo di carta li implementa in modo diverso: una carta numerica vale il proprio valore facciale, una carta azione vale 20, una Wild vale 50. Allo stesso modo, le regole di giocabilità variano tra numeriche, azione e jolly.

Java

```
public abstract class Card {
    protected CardColor color;
    protected CardValue value;

    protected Card(CardColor color, CardValue value) {
        this.color = color;
        this.value = value;
    }

    public abstract int getPoints();
    public abstract boolean isPlayable(Card topCard, CardColor
currentColor);
    public String getImageFileName() {
```

```
        return color + "_" + value.getFileName() + ".JPG";
    }
```

La classe Player segue lo stesso schema. Tutta la gestione della mano di carte, dei punti accumulati e della dichiarazione UNO è implementata nella classe base e condivisa da tutti i giocatori. L'unica differenza tra HumanPlayer e BotPlayer è il metodo astratto isBot().

```
Java
public abstract class Player {
    protected String name;
    protected List<Card> hand;
    protected int points;
    protected boolean unoCalled;

    protected Player(String name) { ... }

    public abstract boolean isBot();
}
```

HumanPlayer non aggiunge alcuna logica propria. BotPlayer invece aggiunge il campo strategy e i metodi di delega, trattati nella [sezione 5](#).

3.2 Interfacce

Il progetto usa un'interfaccia per modellare il comportamento variabile dei bot: BotStrategy.

```
Java
public interface BotStrategy {

    Card chooseCard(List<Card> hand, Card topCard, CardColor currentColor);
    CardColor chooseColor(List<Card> hand, CardColor currentColor);

    boolean shouldPlayDrawnCard(List<Card> hand, Card topCard, CardColor currentColor);
    boolean shouldChallenge(List<Card> hand, int targetHand);
    boolean shouldContestUno();
}
```


Il controller crea il bot con la strategia scelta dall'utente nel menu:

```
Java
case "Bot Casuale":
    player = new BotPlayer(name, new RandomBotStrategy());
    break;
case "Bot Greedy":
    player = new BotPlayer(name, new GreedyBotStrategy());
    break;
```

Aggiungere un nuovo profilo comportamentale del bot richiede solo di implementare BotStrategy, aggiungere un caso nel controller ed aggiornare la view Menu, non è richiesta alcuna modifica ad altre classi.

3.3 Enumerazioni

Il progetto usa quattro enum: GameState, GameMode, CardColor e CardValue.

GameState definisce i sei stati in cui può trovarsi la partita. Ogni metodo pubblico di GameEngine lo usa come preconditione:

```
Java

public void drawDuringTurn(Player player) {
    if (gameState != GameState.WAITING_FOR_PLAYER_ACTION)
        throw new IllegalStateException("Non puoi pescare una carta al momento!");
}
```

Il Controller legge lo stato dopo ogni azione e agisce di conseguenza.

```
Java

public void handleStateChange() {
    gameView.updateTable(engine);
}
```

```

switch (engine.getGameState()) {
case NOT_STARTED:
case WAITING_FOR_PLAYER_ACTION:
case WAITING_FOR_COLOR_CHOICE:
case WAITING_FOR_CHALLENGE:
case ROUND_OVER:
case GAME_OVER:
}

```

GameMode distingue le due modalità di partita. Viene letto in calculateRoundScore() per decidere se la partita termina o continua al round successivo.

CardColor e CardValue modellano i colori e i valori delle carte. CardValue contiene un campo fileName, usato per costruire il nome del file immagine di ogni carta, e per ricavare il valore di una carta numerica.

3.4 Lambda, Comparatori

Nel controller, ogni bottone della View riceve una lambda come EventHandler:

```

Java
menuView.getButtonSingleGame().setOnAction(e -> {
    engine.setGameMode(GameMode.SINGLE_GAME);
    menuView.hideThreshold();
    menuView.showSectionPlayers();
});

```

Nella gestione della mano del giocatore, ogni carta riceve una lambda dedicata che cattura il riferimento alla carta specifica, permettendo così al click su una carta di sapere quale carta giocare:

```

Java
for (Card c : engine.getCurrentPlayer().getHand()) {
    gameView.addCardToHand(c, () -> {
        try {
            engine.playCard(engine.getCurrentPlayer(), c);
            handleStateChange();
        } catch (Exception ex) {
            gameView.showError(ex.getMessage());
        }
    })
}

```

```
});
}
```

In GreedyBotStrategy, la carta da giocare viene selezionata cercando il massimo tra quelle giocabili rispetto al punteggio.

```
Java
hand.stream()
    .filter(c -> c.isPlayable(topCard, currentColor))
    .max(Comparator.comparingInt(Card::getPoints))
    .orElse(null);
```

La scelta del colore più frequente in mano combina groupingBy() e counting() per costruire una mappa, dalla quale viene estratto il massimo:

```
Java
CardColor mostFrequentColor = hand.stream().filter(c -> c.getColor() !=
CardColor.WILD)
    .collect(Collectors.groupingBy(Card::getColor,
Collectors.counting())).entrySet().stream()
    .max(Comparator.comparingLong(entry -> entry.getValue()))
    .map(entry -> entry.getKey()).orElse(colors[random.nextInt(4)]);
```

3.5 Gestione errori

Il progetto non definisce eccezioni personalizzate, utilizzando invece due eccezioni standard di Java più appropriate al contesto.

IllegalStateException viene lanciata quando un'operazione non è permessa nello stato corrente della partita, ad esempio tentare di pescare fuori dal proprio turno o passare il turno senza aver prima pescato o giocato.

```
Java
public void passTurn() {
    if (gameState != GameState.WAITING_FOR_PLAYER_ACTION) {
        throw new IllegalStateException("Non puoi skippare il
turno!");
    }
}
```

```

        if (!madePlay) {
            throw new IllegalStateException("Devi Pescare o Giocare una
            Carta!");
        }
        moveToNextPlayer();
    }
}

```

`IllegalArgumentException` viene usata quando il parametro passato non è valido, ad esempio tentare di giocare una carta non giocabile o aggiungere un giocatore con un nome già in uso.

```

Java
if (!card.isPlayable(getTopCard(), currentColor)) {
    throw new IllegalArgumentException("Carta non giocabile!");
}

```

Le eccezioni lanciate dal model vengono intercettate dal controller con blocchi try-catch e mostrate all'utente tramite la view.

```

Java
gameView.getButtonPass().setOnAction(e -> {
    try {
        engine.passTurn();
        handleStateChange();
    } catch (Exception ex) {
        gameView.showError(ex.getMessage());
    }
});

```

4. Interfaccia Grafica (GUI)

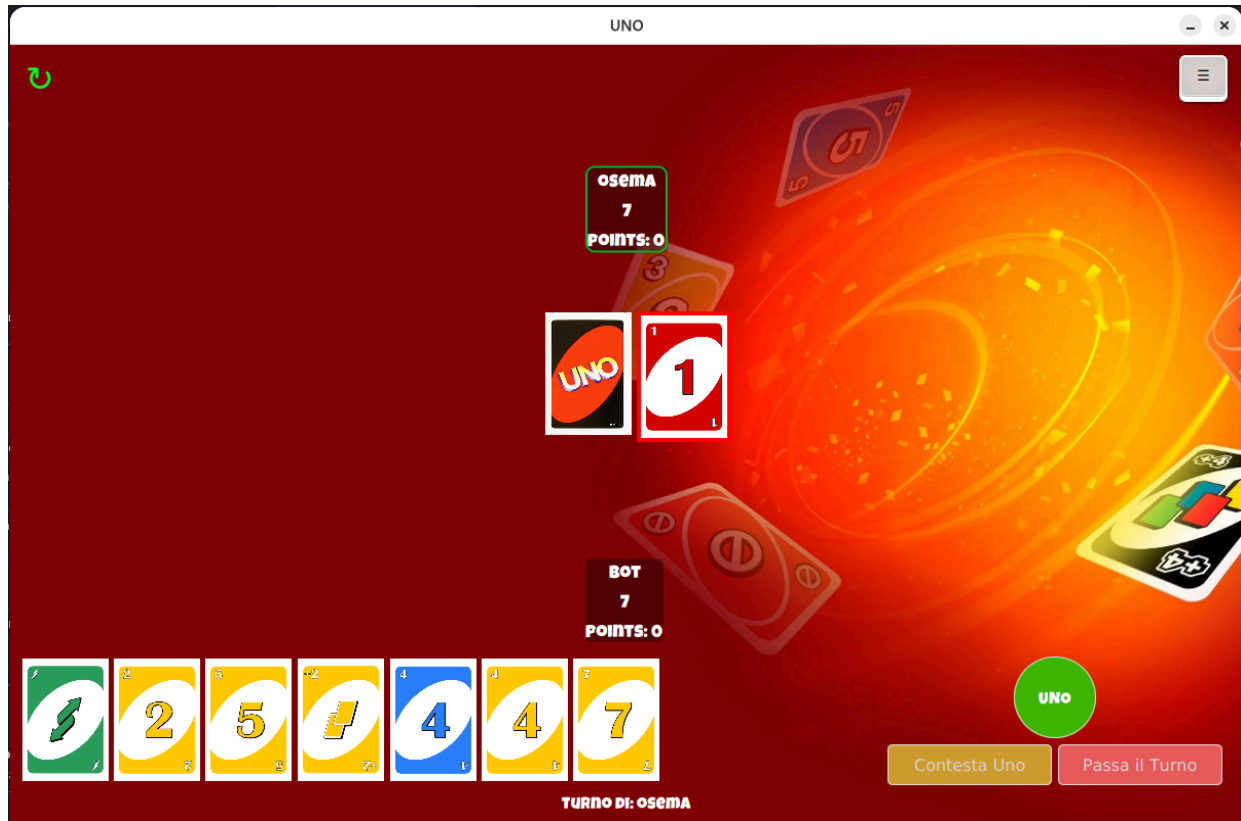
L'interfaccia grafica è realizzata in JavaFX ed è suddivisa in due schermate principali: la schermata di configurazione, che permette di impostare la partita, e la schermata di gioco, che contiene il tavolo di gioco e tutte le interazioni durante la partita.

4.1 Menu principale



4.2 Schermata di gioco

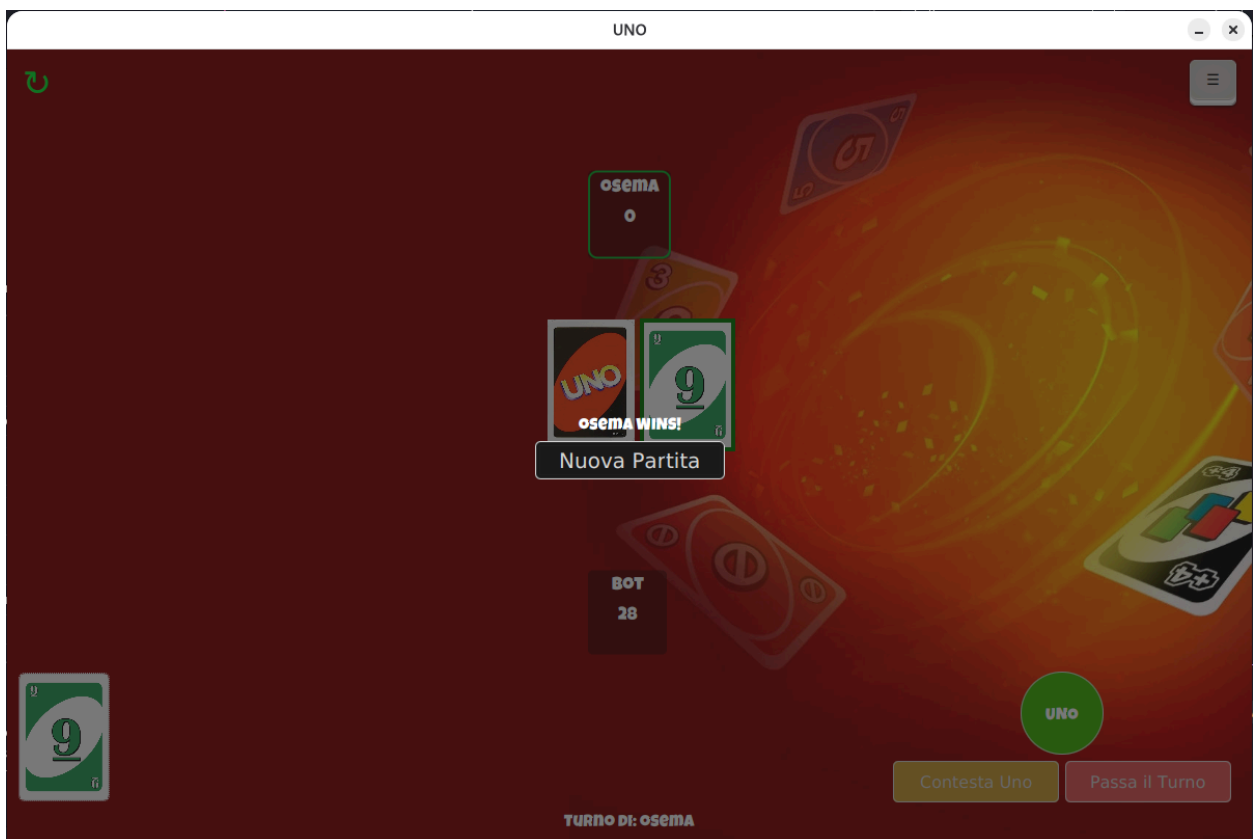
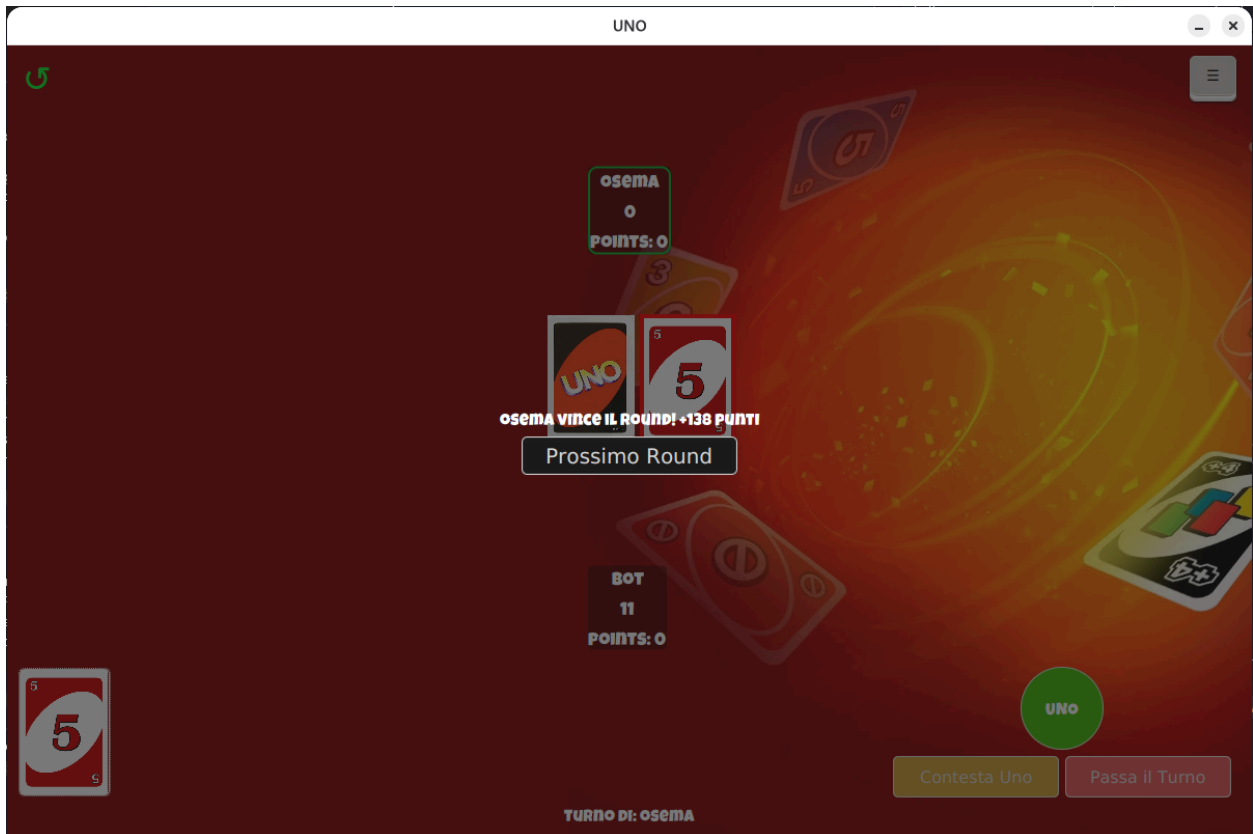
La schermata di gioco mostra il tavolo circolare con i giocatori disposti attorno, il numero di carte di ciascuno e il colore attivo (indicato dal bordo della carta in cima alla pila degli scarti). La mano del giocatore corrente appare nella barra in basso, scorrevole orizzontalmente. I bottoni UNO, Contesta UNO e Passa il Turno sono raggruppati in basso a destra. Il verso di gioco è indicato dalla freccia in alto a sinistra.



4.3 Fasi speciali

Alcune fasi della partita richiedono un'interazione aggiuntiva mostrata tramite overlay sovrapposti al tavolo. Quando un giocatore gioca una carta Wild, appare la selezione del colore; quando viene giocata una Wild Draw Four, il giocatore successivo può scegliere se effettuare il challenge o accettare la penalità.

4.4 Fine round e fine partita



5. Strategie dei Bot

Il progetto implementa due profili comportamentali per i bot, selezionabili nella configurazione iniziale della partita. Entrambi implementano l'interfaccia BotStrategy e vengono iniettati in BotPlayer alla creazione.

Java

```
public class BotPlayer extends Player {
    private BotStrategy strategy;

    public Card chooseCardPlay(Card topCard, CardColor currentColor) {
        return strategy.chooseCard(this.hand, topCard, currentColor);
    }
}
```

5.1 Bot Casuale (RandomBotStrategy)

Il Bot Casuale prende ogni decisione in modo casuale tra le opzioni disponibili, senza alcuna logica strategica.

Java

```
public Card chooseCard(List<Card> hand, Card topCard, CardColor
currentColor) {
    List<Card> playable = hand.stream().filter(c ->
c.isPlayable(topCard, currentColor)).collect(Collectors.toList());

    return playable.isEmpty() ? null :
playable.get(random.nextInt(playable.size()));
}
```

5.2 Bot Greedy (GreedyBotStrategy)

Il Bot Greedy adotta una strategia orientata a massimizzare il valore delle carte giocate, riducendo il proprio punteggio potenziale in caso di sconfitta.

Questa strategia gioca sempre la carta con il punteggio più alto. Questo significa che Wild Draw Four (50 pt) e Wild (50 pt) vengono preferite alle carte azione (20 pt), che a loro volta vengono preferite alle carte numeriche.

Java

```
public Card chooseCard(List<Card> hand, Card topCard, CardColor
currentColor) {
    Card playable = hand.stream().filter(c -> c.isPlayable(topCard,
currentColor)).max(Comparator.comparingInt(Card::getPoints)).orElse(null);

    return playable;
}
```

Quando gioca una Wild, sceglie il colore più frequente nella propria mano.

Se giocabile, gioca sempre la carta pescata. Contesta sempre un avversario che non ha dichiarato UNO. Effettua casualmente il challenge Wild Draw Four.

6. Istruzioni per l'uso

Prerequisiti

- Java 21 o superiore
- Apache Maven 3.9.9

Da terminale

Shell

```
mvn javafx:run
```

Da Eclipse

- Run -> Run Configurations
- Tasto destro su Maven Build -> New Configuration
- In Goals inserire javafx:run
- Cliccare Run

7. Limiti Noti e Conclusioni

Il processo di sviluppo è risultato abbastanza lineare, in quanto dall'inizio si è avuto una idea di come procedere. Durante la fase di implementazione si è reso necessario un intervento di refactoring per migliorare il controllo della logica di gioco. È stata introdotta l'enumerazione GameState per la gestione dello stato della partita, garantendo così una validazione di ogni azione eseguita dai giocatori in ogni fase del round.

Il progetto realizza un'implementazione del gioco UNO con interfaccia grafica, supportando tutte le regole della versione base, due modalità di partita, partite miste tra giocatori umani e bot, e due profili comportamentali distinti per i bot. L'architettura MVC adottata garantisce una separazione delle responsabilità, mentre l'uso di pattern, ad esempio Strategy, rende il gioco facilmente estensibile.